



Programmation Python

Filière LSI1



Chapitre 4:

Programmation orientée objet POO en Python

2020-2021 - SEMESTRE 2

nassima.medimegh@fsm.rnu.tn



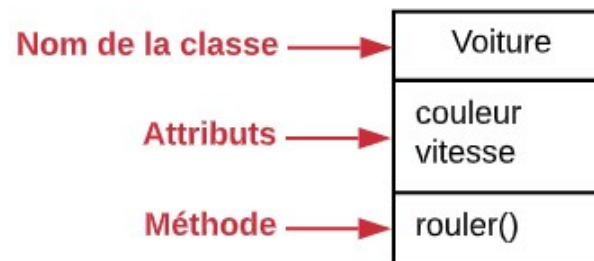
1. Le concept de POO en Python

- La **programmation orientée objet**, ou **POO**, est un **paradigme de programmation** qui permet de structurer les programmes de manière à ce que les propriétés et les comportements soient regroupés dans des objets à part.
- Les objets peuvent interagir entre eux, cela facilite grandement la compréhension du code et sa maintenance.
- Nous avons vu plusieurs types de base en Python (**int** pour les entiers, **float** pour les flottants, **str** pour les chaînes de caractères, etc.). La notion de **classe** va en quelque sorte nous permettre de généraliser la notion de « type » afin de créer de nouvelles structures de données.

python 2. Les classes

Définition d'une classe

- Une **classe** permet de définir un nouveau type permettant de créer des **instances** (des **objets**/variables) contenant des données (**attributs**) et des fonctions (**méthodes**) associées à ces données.



python 2. Les classes

Création d'une classe

- Comme dans tout langage orienté objet, il est nécessaire de définir la manière dont on initialise un objet de cette classe, puis définir les différentes opérations/fonctions permises avec cet objet.
- Pour créer une classe en Python, on utilise l'instruction

`class nom_de_la_classe :`

- Par convention en Python, le nom identifiant d'une classe débute par une majuscule.

`class Point():`

`"Definition d'un point geometrique"`

python 2. Les classes

La méthode « constructeur »

- On crée ensuite une méthode qui permet de construire les objets, appelé **constructeur** via l'instruction : `def __init__(self):`
- La méthode `__init__()` est appelée lors de la création d'un objet.
- Pour accéder aux données d'un objet dans une de ses méthodes, on utilise le mot clef `self`.

```
class Point():  
    "Definition d'un point geometrique"  
    def __init__(self,x,y):  
        self.abscisse = x  
        self.ordonnée=y
```

- `self.abscisse` est une manière de stocker une information dans la classe. On parle d'attribut de classe. Dans notre cas, on stock la valeur x dans l'attribut abscisse .

python 2. Les classes

Les objets

- Un objet est une **instance** d'une classe. On peut créer autant d'objets que l'on désire avec une classe .

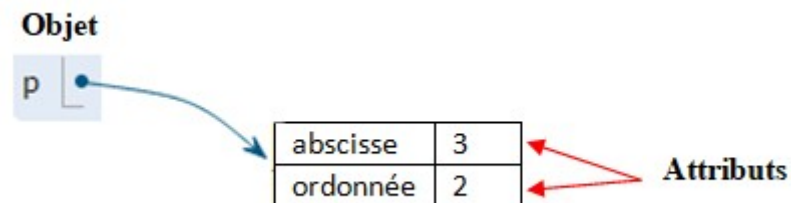
```
>>> P=Point(3,2)
```

Les attributs de classe

- Les attributs de classe permettent de stocker des informations au niveau de la classe. Elle sont similaires aux variables.

```
>>> P. abscisse
```

```
3
```



Les méthodes

- Une **méthode de classe** est une **fonction** ou **procédure** définie au sein de la classe, permettant de définir des propriétés ou comportements des objets d'instances.



2. Les classes

Exemple

```
class Point():  
    "Definition d'un point geometrique"  
    def __init__(self,x,y):  
        self.abscisse=x  
        self.ordonnée=y  
    def deplace(self, dx, dy):  
        self.abscisse = self.abscisse + dx  
        self.ordonnée = self.ordonnée + dy  
  
a = Point(2,3)  
print("a : x =", a.abscisse, "y =", a.ordonnée)  
a.deplace(3, 5)  
print("a : x =", a.abscisse, "y =", a.ordonnée)
```

a : x = 2 y = 3
a : x = 5 y = 8

Exercice 1

- Définir une classe **Point3D** qui sera analogue à la classe **Point** mais pour des points dans l'espace à 3 dimensions.
- Créer deux objets de type **Point3D** qui seront référencés par les variables **A** et **B**. Initialiser ces points et afficher leurs coordonnées x, y, z .
- Ajouter une méthode **Distance()** qui calcule la distance euclidienne entre deux points



3. L'héritage de classe

- Comme tous les langages objets, Python accepte l'héritage, pour éviter de recopier le code d'une classe.
- La méthode d'héritage consiste à créer à partir d'une **classe parent** ou (**classe mère**) une autre classe appelé **classe fille** ou (**classe enfant**) qui hérite toutes les méthodes et attributs de la classe mère.
- La syntaxe générale de l'héritage :
`class classe_fille(classe_mère) :`



3. L'héritage de classe

Exemples d'héritages en Python

```
class Personne:
    def __init__(self,nom,age):
        self.nom = nom
        self.age=age
# La classe fille Student hérite de la classe mère Personne
class Student(Personne):
    # définition des attributs
    def __init__(self,nom,age,filiere):
        # héritage des attributs depuis la classe mère Personne
        Personne.__init__(self,nom,age)
        # ajout d'un nouvel attribut filiere à la classe fille
        self.filiere = filiere
Stud = Student("Ahmed",25,"informatique")
print("Le nom de l'étudiant est : ",Stud.nom)
print("L'age de l'étudiant est : ",Stud.age)
print("La filière de l'étudiant est : ",Stud.filiere)
```

Le nom de l'etudiant est : Ahmed
L'age de l'etudiant est : 25
La filiere de l'etudiant est : Informatique



3. L'héritage de classe

- Si une classe hérite d'une autre classe, elle hérite aussi les méthodes de son parent.

```
class Personne:
    def __init__(self,nom,age):
        self.nom = nom
        self.age=age
    def afficher(self):
        print(" l'age de " ,self.nom, " est: ", self.age)
# La classe fille Student hérite de la classe mère Personne
class Student(Personne):
    # définition des attributs
    def __init__(self,nom,age):
        # héritage des attributs depuis la classe mère
        Personne.__init__(self,nom,age)
Stud = Student("Ahmed",25)
Stud.afficher()
```

→ l'age de Ahmed est: 25



3. L'héritage de classe

Exercice 2

- Créer une classe **Triangle3D** qui hérite les attributs de la classe **Point3D** de l'exercice 1.
- Définir dans cette classe un triangle par 3 points 3D